

Rust for Safety- critical systems

David Fernando Bolaños López - U0302313

Ángela Nistal Guerrero - U0301919

Introducción



Podcast

Podcast SE Radio 690: Florian Gilcher

Cofundador de Ferrous System y de la Rust
Foundation



Importancia

Desarrollar software para entornos misión-
critical y safety-critical

Ventajas de Rust

ÍNDICE

01

Criticidad de
un proyecto

02

Uso de Rust

03

Rust vs C/C++

04

Ferrocene

05

Limitaciones
Casos de uso

06

Futuro



01

Criticidad del proyecto

Criticidad del proyecto



Mission-critical

Sistemas cuyo fallo puede provocar pérdidas económicas graves o caídas en servicios e infraestructuras



Safety-critical

Sistemas cuyo fallo puede poner en riesgo vidas humanas.



Mixto

Hoy en día ambos se mezclan.



02

Uso de Rust

Uso de Rust para sistemas críticos

“you don’t need to buy a tool for proving memory safety on Rust because the compiler already does it.” — Florian Gilcher

- **Ownership**: control claro sobre qué parte del programa posee cada recurso.
- **Borrowing**: referencias seguras con tiempos de vida rastreados por el compilador.
- **Código nativo**: ejecutables eficientes sin un runtime pesado



03

Rust vs C/C++

	Rust	C/C++:
Seguridad en memoria	Compilador en tiempo de compilación	Bugs, herramientas externas costosas
Verificación	Detectados en compilación	Necesarias pruebas manuales y pruebas para post-compilación
Ecosistema	En crecimiento	Maduro (perfeccionando)
Concurrencia	Previene ciertas condiciones de carrera limitando el <code>aliasing mutable</code>	Mayor grado de libertad (errores)

Empresas que mejoraron su productividad al usar Rust: Google y Volvo.



04

Ferrocene

Ferrocene: Toolchain Certificada

Diseñada para sistemas safety-critical:

- Cumple ISO 26262, IEC 61508
- Compilador Rust auditado y con trazabilidad completa
- Evidencia documentada de cada optimización
- Soporte para bare-metal
- Herramientas de verificación integradas

Importancia:

- Reutilización de evidencia ya certificada
- Reducción drástica de pruebas y costes
- Aceleración de la homologación
- Especialmente valioso en automoción y aviación, donde certificar puede llevar años





05

Limitaciones Casos de uso

Limitaciones

En entornos críticos hay que tratar la toolchain como parte del sistema:

- Desactivar compilación incremental en builds finales.
- 'unsafe' debe estar justificado y revisado.
- La certificación requiere procesos de calidad adicionales.

Casos de uso

Tock OS

Sistema operativo en Rust, usado por Google; en proceso de certificación

Dispositivo médico

Implementado en un mes cumpliendo requisitos. Actualmente está en evaluación regulatoria

Sudo

Versión moderna de Unix. Reescritura en Rust → más compacto, rápido y seguro; se detectaron errores en la versión en C

Android (Google)

Integración masiva de Rust en Android; reportan el doble de productividad

Volvo

En proyectos mixtos C/Rust lograron 2-4x más velocidad de desarrollo





06

Futuro

Futuro de Rust

La adopción de Rust en sistemas safety-critical se llevará a cabo cuando mejore la seguridad de memoria o reduzca riesgos.

Rust puede llegar a convertirse en un lenguaje de referencia para proyectos memory safety.

No plantea un reemplazo, sino una evolución del sector.

¡ Gracias !

¿ Alguna pregunta ?